COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	October 20, 2026	**DATE SUBMITTED:**	October 20, 2026
NAME:	Dizor, Elijah Gresola, Aryanna Keith G	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 14 **Algorithm Complexity**

1. PROCEDURE OUTPUT

	BEST CASE	WORST CASE	ANALYSIS
Bubble Sort	O(n)	O(n^2)	Its performance depends on the arrangement of the data. If the elements are already sorted, it can finish in O(n) time. However, if the elements are in reverse order, it takes O(n^2) time because it needs more comparisons and swaps.
Merge Sort	O(n log n)	O(n log n)	It divides the data into smaller parts and merges them in sorted order. Its time complexity remains O(n log n) in both the best and worst cases, making it more efficient for large datasets, although it requires additional memory.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	Bubble Sort	Merge Sort

Best Case	100:
	`Bubble Sort: 0.600 microseconds`
	1000:
	`Bubble Sort: 4.000 microseconds`
	10000:
	`Bubble Sort: 39.400 microseconds`
		100:
		`Merge Sort: 10.200 microseconds`
		1000:
		`Merge Sort: 136.500 microseconds`
		10000:
		`Merge Sort: 1637.800 microseconds`

Worst Case	100: `Bubble Sort: 58.200 microseconds`	100: `Merge Sort: 9.500 microseconds`

	1000:
	`Bubble Sort: 5492.000 microseconds`
	
	10000:
	`Bubble Sort: 596316.900 microseconds`
		1000:
		`Merge Sort:  117.500 microseconds`
		
		10000:
		`Merge Sort:  1569.300 microseconds`

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

ANALYSIS
Based on the results, Bubble Sort was faster than Merge Sort when the data was already sorted. However, when the data was in reverse order, Bubble Sort became much slower as the input size increased. For 10,000 elements, Bubble Sort took 596,316.900 microseconds, while Merge Sort took only 1,569.300 microseconds. This shows that Merge Sort is more efficient for large datasets in the worst case.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be theoretical and empirical analysis.

Empirical Analysis

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	`0.065 microseconds`	`2.825 microseconds`
10,000	`0.087 microseconds`	`27.351 microseconds`
10,0000	`0.129 microseconds`	`297.294 microseconds`

Time Complexity

	Binary Search	Linear Search
Best-Case	O(1)	O(1)
Worst-Case	O(log n)	O(n)

Space Complexity

Binary Search	Linear Search
O(1)	O(1)

ANALYSIS

Based on the results, Binary Search was faster than Linear Search for all input sizes. As the input size increased from 1,000 to 100,000, the execution time of Linear Search increased from 2.825 to 297.294 microseconds, while Binary Search only increased from 0.065 to 0.129 microseconds. This is because Linear Search checks each element one by one, while Binary Search repeatedly divides the search range in half. Therefore, Binary Search is more efficient for large datasets, provided that the array is sorted.

CONCLUSION/LEARNINGS

DIZOR	Algorithmic complexity counts the number of operations that the algorithm performs relative to the input size. Understanding time complexities allows you to predict how your applications will behave and know the runtimes before deploying it. Runtimes are usually affected by computers used but hardware cannot change the mathematical structure of an algorithm, because it is based on what was applied by the coder.
GRESOLA	These activities helped me understand how the efficiency of an algorithm can affect its performance when working with different input sizes. I learned that Bubble Sort and Linear Search can take longer as the amount of data increases, while Merge Sort and Binary Search offer better efficiency for larger datasets. I also realized that choosing an algorithm depends on the problem, the arrangement of the data, and the resources needed. By comparing their execution times and complexities, I gained a better understanding of why selecting the right algorithm is important in programming.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar correction, code assistance, and explanation of concepts.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: ChatGPT was used for grammar improvement, code assistance, and explaining concepts I did not understand. It did not replace my own thinking, analysis, authorship, or completion of the required work.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.